

Documentation for `advanceSQL.php`

Overview

`advanceSQL.php` is a custom PHP utility file that provides advanced MySQL database helper functions using the `mysqli` extension. It simplifies CRUD (Create, Read, Update, Delete) operations and supports:

- Dynamic SQL query building
- Prepared statements
- Transaction handling
- Advanced filtering and searching
- Pagination and ordering
- Secure parameter binding

The file is mainly designed to reduce repetitive SQL code in PHP projects.

Main Functions

1. `advanceSelect()`

Purpose

Retrieves records from a database table with advanced filtering options.

Syntax

```
advanceSelect($table, $selectable='*', $condition=[], $conn=null)
```

Parameters

Parameter	Description
<code>\$table</code>	Database table name
<code>\$selectable</code>	Columns to select
<code>\$condition</code>	Conditions and filters
<code>\$conn</code>	MySQL connection object

Supported Conditions

Basic Conditions

```
[
  'id' => 1,
  'status' => 'active'
]
```

Less Than or Equal (`__LESSER`)

```
[
  '__LESSER' => [
    'price' => 100
  ]
]
```

Greater Than or Equal (`__GREATER`)

```
[
  '__GREATER' => [
    'age' => 18
  ]
]
```

Between Range (`__BETWEEN`)

```
[
  '__BETWEEN' => [
    'salary' => [1000, 5000]
  ]
]
```

Search Conditions (`__SEARCH`)

```
[
  '__SEARCH' => [
    '__CONJUNCTOR' => 'OR',
    'name' => "LIKE '%John'",
    'email' => "LIKE '%gmail'"
  ]
]
```

```
]
]
```

Ordering

```
[
  '__ORDERBY' => 'id',
  '__ASC' => 'DESC'
]
```

Pagination

```
[
  '__LIMIT' => 10,
  '__OFFSET' => 20
]
```

Example

```
list($err, $data) = advanceSelect(
    'users',
    '*',
    [
        'status' => 'active',
        '__LIMIT' => 5
    ]
);
```

Return Value

```
[error, data, connection]
```

2. `advanceSelect2()`

Purpose

Performs secure SELECT queries using prepared statements.

Syntax

```
advanceSelect2($table, $selectable='*', $condition=[], $conn=null)
```

Features

- Uses parameter binding
- Prevents SQL injection
- Safer than manual query building

Example

```
list($err, $result) = advanceSelect2(  
    'users',  
    '*',  
    ['id' => 1]  
);
```

3. advanceUpdate()

Purpose

Updates records in a table.

Syntax

```
advanceUpdate($table, $param, $condition, $conn=null)
```

Parameters

Parameter	Description
<code>\$table</code>	Table name
<code>\$param</code>	Columns and values to update
<code>\$condition</code>	WHERE conditions

Example

```
advanceUpdate(  
    'users',  
    ['name' => 'John'],  
    ['id' => 1]  
);
```

Return

```
[error, affected_rows, connection]
```

4. advanceUpdate2()

Purpose

Updates records using prepared statements.

Advantages

- More secure
- Supports parameter binding
- Reduces SQL injection risk

Example

```
advanceUpdate2(  
    'users',  
    ['name' => 'John'],  
    ['id' => 1]  
);
```

5. advanceInsert()

Purpose

Inserts a new row into a table.

Syntax

```
advanceInsert($table, $param, $conn=null)
```

Example

```
advanceInsert(  
    'users',  
    [  
        'name' => 'John',  
        'email' => 'john@gmail.com'  
    ]  
);
```

Return

```
[error, inserted_id, connection]
```

6. advanceInsert2()

Purpose

Performs INSERT operations using prepared statements.

Features

- Secure insertion
- Uses placeholders (?)
- Parameter binding

Example

```
advanceInsert2(  
    'users',  
    [  
        'name' => 'John',  
        'email' => 'john@gmail.com'  
    ]  
);
```

```
]
);
```

7. `advanceDelete()`

Purpose

Deletes rows from a table.

Syntax

```
advanceDelete($table, $condition=[], $conn=null)
```

Example

```
advanceDelete(
    'users',
    ['id' => 1]
);
```

Return

```
[error, affected_rows, connection]
```

8. `advanceDelete2()`

Purpose

Deletes records securely using prepared statements.

Example

```
advanceDelete2(
    'users',
```

```
        ['id' => 1]  
    );
```

9. mysqli_execute_query2()

Purpose

Custom helper function for executing prepared statements.

Syntax

```
mysqli_execute_query2(mysqli $conn, string $query, array|null $param=null)
```

Features

- Automatically detects parameter types
- Executes prepared statements
- Returns:
 - Selected rows
 - Insert ID
 - Affected rows

Supported Parameter Types

Type	Identifier
Integer	<code>i</code>
Double	<code>d</code>
String	<code>s</code>
Blob	<code>b</code>

Transaction Handling

The file supports transaction management using:

```
mysqli_begin_transaction()  
mysqli_commit()  
mysqli_rollback()
```

If any query fails:

- Transaction rolls back automatically
- Error is returned
- Remaining queries stop executing

Security Features

SQL Injection Prevention

The file improves security by:

- Escaping user inputs using `mysqli_real_escape_string()`
- Using prepared statements
- Binding parameters dynamically

Recommended Functions

More secure versions:

Operation	Recommended Function
Select	<code>advanceSelect2()</code>
Insert	<code>advanceInsert2()</code>
Update	<code>advanceUpdate2()</code>
Delete	<code>advanceDelete2()</code>

Advantages of This Utility

- Reduces repetitive SQL code
- Easier CRUD operations
- Supports advanced filtering
- Supports pagination and ordering
- Better security with prepared statements
- Centralized database logic

- Easier maintenance
-

Limitations

- Some functions still build raw SQL strings manually
 - Limited validation of table/column names
 - Complex joins are not directly supported
 - Error handling uses `die()` in some areas
-

Suggested Improvements

1. Add JOIN support
 2. Add GROUP BY support
 3. Improve exception handling
 4. Add query logging
 5. Add ORM-style model support
 6. Add support for transactions across multiple functions
 7. Improve type validation
-

Conclusion

`advanceSQL.php` is a lightweight PHP database abstraction helper that simplifies database operations using MySQLi. It provides both manual query building and prepared-statement-based operations for safer and more flexible database management.

The utility is useful for small to medium PHP applications that need reusable CRUD operations without implementing a full ORM framework.

Source file: [advanceSQL.php GitHub Source](#)